GCP-ACE Advance — Part II

Deep-Dive Theory & Hands-On Lab Solutions (Sections 1–23)

Associate Cloud Engineer — Advanced Lab Guide

July 2026

Contents

Introduction **4**

Section 1: Deploying a Google Kubernetes Engine Cluster **5**
Theory . . . 5
Hands-On Lab: Deploy a GKE Cluster and a Pod . . . 5

Section 2: Create Kubernetes Cluster from Google Cloud Console **7**
Theory . . . 7
Hands-On Lab . . . 7

Section 3: Deploying and Implementing App Engine, Cloud Run, and Cloud Functions Resources **9**
Theory . . . 9
Hands-On Lab . . . 9

Section 4: Google Cloud Function Hands-On — Java Spring Boot + GKE + Pub/Sub Trigger (HTTP, Cloud Storage, Pub/Sub) **12**
Theory . . . 12
Hands-On Lab . . . 12

Section 5: Deploying and Implementing Data Solutions **15**
Theory . . . 15
Hands-On Lab . . . 15

Section 6: Create Bucket & Upload (Image) Object — Java Spring Boot + GKE **17**
Theory . . . 17
Hands-On Lab . . . 17

Section 7: Labs — Cloud Spanner, Cloud Memorystore, Cloud Firestore, Cloud Bigtable **24**
Theory . . . 24
Hands-On Lab . . . 24

Section 8: Cloud Dataflow **26**
Theory . . . 26
Hands-On Lab . . . 26

Section 9: Cloud Dataproc **28**

Theory . . . 28
Hands-On Lab — Demo: Create a Cluster and Submit a Spark Job to Calculate π . . . 28

Section 10: Cloud Pub/Sub (Hands-On) 30
Theory . . . 30
Hands-On Lab . . . 30

Section 11: Deploying and Implementing Networking Resources 32
Theory . . . 32
Hands-On Lab . . . 32

Section 12: Organization with Project1 and Project2 — Host Project using Shared VPC and VPC Peering (Hands-On) 34
Theory . . . 34
Hands-On Lab . . . 34

Section 13: Managing Google Kubernetes Engine Resources 36
Theory . . . 36
Hands-On Lab . . . 36

Section 14: Managing App Engine and Cloud Run Resources 38
Theory . . . 38
Hands-On Lab . . . 38

Section 15: Managing Storage and Database Solutions 39
Theory . . . 39
Hands-On Lab . . . 39

Section 16: Managing Networking Resources 41
Theory . . . 41
Hands-On Lab . . . 41

Section 17: Monitoring and Logging 42
Theory . . . 42
Hands-On Lab . . . 42

Section 18: Uptime Check & Alerting (End-to-End Lab) 44
Theory . . . 44
Hands-On Lab . . . 44

Section 19: Lab — Cloud Logging (Hands-On) 46
Theory . . . 46
Hands-On Lab . . . 46

Section 20: Cloud Debugger 47
Theory . . . 47
Hands-On Lab . . . 47

Section 21: Managing Identity and Access Management (IAM) 49
Theory . . . 49
Hands-On Lab . . . 49

Section 22: Managing Service Accounts 51
Theory . . . 51
Hands-On Lab . . . 51

Section 23: Consolidated Service Account Lab — Limited Privileges, VM Assignment, Cross-Project Access **53**
Theory 53
Hands-On Lab 53

Introduction

This guide is a deep-dive companion for the **GCP-ACE (Google Cloud Associate Cloud Engineer) Advance — Part II** curriculum. Each of the 23 sections below includes:

- **Theory** — the underlying concept, why it exists, and where it fits in the GCP ecosystem.
- **Hands-On Lab** — step-by-step `gcloud` CLI commands and/or Cloud Console click-paths you can execute directly in a GCP project.

  Prerequisites: A GCP project with billing enabled, `gcloud` CLI installed and authenticated (`gcloud init`), and the following APIs enabled as you go: Compute Engine, GKE, Cloud Functions, Cloud Run, App Engine, Cloud SQL, BigQuery, Spanner, Pub/Sub, Bigtable, Dataproc, Dataflow, Cloud Storage, Cloud Monitoring/Logging, and IAM.

Section 1: Deploying a Google Kubernetes Engine Cluster

Theory

Google Kubernetes Engine (GKE) is Google's managed Kubernetes service. It handles the control plane (API server, etcd, scheduler, controller manager) for you, so you only manage worker nodes (and in **Autopilot** mode, not even that). Key concepts:

- **Cluster**: A set of node machines running Kubernetes, managed by a control plane.
- **Node / Node Pool**: A node is a VM (Compute Engine instance) running kubelet. A node pool is a group of nodes with the same configuration (machine type, disk, labels, taints).
- **Pod**: The smallest deployable unit — one or more containers sharing network/storage.
- **Deployment**: Manages a ReplicaSet of Pods, enabling rolling updates and self-healing.
- **Service**: A stable network endpoint (ClusterIP, NodePort, or LoadBalancer) in front of a set of Pods.
- **Standard vs Autopilot mode**: Standard gives you full node control (you choose machine types, manage node pools); Autopilot is fully managed and bills per Pod resource request.
- **Monitoring & Logging**: GKE integrates natively with **Cloud Monitoring** and **Cloud Logging** — enabled by default on cluster creation, streaming node, pod, and container-level metrics/logs.

Hands-On Lab: Deploy a GKE Cluster and a Pod

Step 1 — Enable required APIs

```
gcloud services enable container.googleapis.com compute.googleapis.com
```

Step 2 — Set default project/zone

```
gcloud config set project YOUR_PROJECT_ID
gcloud config set compute/zone asia-south1-a
```

Step 3 — Create a GKE Standard cluster

```
gcloud container clusters create ace-demo-cluster \
  --zone asia-south1-a \
  --num-nodes 3 \
  --machine-type e2-medium \
  --enable-stackdriver-kubernetes
```

Step 4 — Get cluster credentials for kubectl

```
gcloud container clusters get-credentials ace-demo-cluster --zone asia-south1-a
```

Step 5 — Deploy a container application as a Pod

```
kubectl run hello-app --image=gcr.io/google-samples/hello-app:1.0 --port=8080
```

Step 6 — Verify the Pod

```
kubectl get pods
kubectl describe pod hello-app
```

Step 7 — Expose the Pod as a Service

```
kubectl expose pod hello-app --type=LoadBalancer --port=80 --target-port=8080 --name=hello-app-
kubectl get service hello-app-svc
```

Wait for `EXTERNAL-IP` to populate, then browse to `http://<EXTERNAL-IP>`.

Step 8 — Configure Monitoring & Logging (if not enabled at creation)

```
gcloud container clusters update ace-demo-cluster \
  --zone asia-south1-a \
  --enable-stackdriver-kubernetes
```

In **Console → Kubernetes Engine → Clusters → ace-demo-cluster**, open the **Observability** tab to view CPU/Memory dashboards, and **Logs Explorer** to filter:

```
resource.type="k8s_container"
resource.labels.cluster_name="ace-demo-cluster"
```

Cleanup

```
kubectl delete service hello-app-svc
kubectl delete pod hello-app
gcloud container clusters delete ace-demo-cluster --zone asia-south1-a
```

Section 2: Create Kubernetes Cluster from Google Cloud Console

Theory

While `gcloud`/`kubectl` are preferred for repeatability and automation, the **Cloud Console** provides a GUI workflow that mirrors the same underlying API calls. The end-to-end workflow for shipping a containerized app to GKE is:

1. **Create the cluster** (console or CLI).
2. **Build a Docker image** of your application.
3. **Push the image** to a registry (**Artifact Registry**, the successor to Container Registry/GCR).
4. **Deploy the image** to the cluster as a Workload (Deployment).
5. **Expose the Workload** as a Service so traffic can reach the Pods.

This mirrors the standard CI/CD path: source → image → registry → deploy → expose.

Hands-On Lab

Step 1 — Create the cluster via Console

Console → Kubernetes Engine → Clusters → Create → GKE Standard - Name: `console-cluster` - Location type: Zonal → `asia-south1-a` - Node pool: 3 nodes, `e2-medium` - Click **Create** and wait ~3–5 minutes for provisioning.

Step 2 — Create a Docker image (Java Spring Boot)

```
mkdir hello-springboot && cd hello-springboot
```

`src/main/java/com/ace/HelloApplication.java`:

```
package com.ace;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
public class HelloApplication {

  @GetMapping("/")
  public String hello() {
    return "Hello from GKE (Java Spring Boot)!";
  }

  public static void main(String[] args) {
    SpringApplication.run(HelloApplication.class, args);
  }
}
```

`application.properties`:

```
server.port=8080
```

`Dockerfile` (multi-stage build):

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -q clean package -DskipTests

FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
COPY --from=build /app/target/hello-springboot-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

```
docker build -t hello-springboot:v1 .
```

Step 3 — Push it to Artifact Registry

```
gcloud artifacts repositories create my-repo \
  --repository-format=docker \
  --location=asia-south1

gcloud auth configure-docker asia-south1-docker.pkg.dev

docker tag hello-springboot:v1 asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/hello-springb
docker push asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/hello-springboot:v1
```

Step 4 — Deploy the Docker image to the cluster

Console → Kubernetes Engine → Workloads → Deploy - Select **Existing container image** → paste the Artifact Registry path. - Application name: `hello-springboot-app` - Namespace: `default` - Cluster: `console-cluster` - Click **Deploy**.

Equivalent CLI:

```
kubectl create deployment hello-springboot-app \
  --image=asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/hello-springboot:v1
```

Step 5 — Expose as a Service

Console → Workloads → hello-springboot-app → Expose - Port: 80, Target port: 8080, Service type: **Load balancer**

Equivalent CLI:

```
kubectl expose deployment hello-springboot-app --type=LoadBalancer --port=80 --target-port=8080
kubectl get service hello-springboot-app
```

Browse to the `EXTERNAL-IP` once assigned to confirm the app is reachable.

Section 3: Deploying and Implementing App Engine, Cloud Run, and Cloud Functions Resources

Theory

These three are GCP's **serverless compute** options, each targeting a different use case:

Service	Best For	Unit of Deployment	Scale-to-zero
App Engine	Full web apps/APIs	Application versions	Standard: yes; Flex: no
Cloud Run	Any containerized service	Container image (revision)	Yes
Cloud Functions	Single-purpose event handlers	Function (source or container)	Yes

Key operational concepts: - **Versions & Traffic Splitting**: App Engine and Cloud Run both let you deploy a new version/revision while keeping the old one live, and split traffic between them by percentage (canary releases, blue/green deployments, A/B testing). - **Scaling configuration**: You can set `min_instances`/`max_instances` to control cold starts vs. cost, and concurrency (Cloud Run) to control how many requests one instance handles simultaneously. - **Event-driven triggers**: Cloud Functions (and Cloud Run via Eventarc) can be invoked directly by **Cloud Pub/Sub** messages, **Cloud Storage** object changes (finalize/delete/archive/metadataUpdate), Firestore changes, or plain **HTTP** requests.

Hands-On Lab

A. App Engine — Deploy a Java Spring Boot App with Traffic Splitting

```
gcloud app create --region=asia-south1

# app.yaml (Java 17 standard runtime, packaged Spring Boot jar as the entrypoint)
cat > app.yaml << 'EOF'
runtime: java17
instance_class: F2
entrypoint: java -jar target/hello-spring-boot-0.0.1.jar
automatic_scaling:
  min_instances: 0
  max_instances: 5
EOF

# Build the Spring Boot jar first
mvn clean package

gcloud app deploy --version=v1 --no-promote
gcloud app deploy --version=v2 --no-promote

# Split traffic 80/20 between v1 and v2
gcloud app services set-traffic default --splits=v1=0.8,v2=0.2
```

B. Cloud Run — Deploy a Java Spring Boot Container, Update Scaling, Traffic Split

```
# Build & push the Spring Boot image (see Section 2 for the Dockerfile pattern, using
# a Java base image such as eclipse-temurin:17-jre and COPY target/*.jar app.jar)
docker build -t hello-spring:v1 .
docker tag hello-spring:v1 asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/hello-spring:v1
docker push asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/hello-spring:v1

gcloud run deploy hello-run \
  --image=asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/hello-spring:v1 \
  --region=asia-south1 \
  --platform=managed \
  --min-instances=0 --max-instances=10 \
  --allow-unauthenticated

# Deploy a new revision without shifting traffic
gcloud run deploy hello-run --image=asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/hello-sp

# Split traffic 90/10 between latest (v1) and the v2 tag
gcloud run services update-traffic hello-run --to-tags v2=10 --to-latest=90
```

C. Cloud Functions — HTTP Trigger (Java, Spring-style Functions Framework)

```
mkdir http-fn && cd http-fn
```

```
package com.ace.functions;

import com.google.cloud.functions.HttpFunction;
import com.google.cloud.functions.HttpRequest;
import com.google.cloud.functions.HttpResponse;
import java.io.BufferedWriter;

public class HelloHttp implements HttpFunction {
  @Override
  public void service(HttpRequest request, HttpResponse response) throws Exception {
    BufferedWriter writer = response.getWriter();
    writer.write("Hello from Cloud Function (HTTP)!");
  }
}
```

Add the `functions-framework-api` dependency to `pom.xml` (see Section 4, Step 1), then deploy:

```
gcloud functions deploy hello-http \
  --gen2 --runtime=java17 --region=asia-south1 \
  --source=. --entry-point=com.ace.functions.HelloHttp \
  --trigger-http --allow-unauthenticated
```

D. Cloud Functions — Cloud Storage Event Trigger (Java)

```
gcloud storage buckets create gs://YOUR_PROJECT_ID-fn-bucket --location=asia-south1
```

```
package com.ace.functions;
```

```java
import com.google.cloud.functions.CloudEventsFunction;
import io.cloudevents.CloudEvent;

public class OnObjectFinalize implements CloudEventsFunction {
  @Override
  public void accept(CloudEvent event) {
    System.out.println("New object event id: " + event.getId() + " type: " + event.getType());
  }
}
```

```
gcloud functions deploy on-storage-upload \
  --gen2 --runtime=java17 --region=asia-south1 \
  --source=. --entry-point=com.ace.functions.OnObjectFinalize \
  --trigger-event-filters="type=google.cloud.storage.object.v1.finalized" \
  --trigger-event-filters="bucket=YOUR_PROJECT_ID-fn-bucket"
```

E. Cloud Functions — Pub/Sub Event Trigger (Java)

```
gcloud pubsub topics create fn-topic
```

```java
package com.ace.functions;

import com.google.cloud.functions.CloudEventsFunction;
import io.cloudevents.CloudEvent;
import java.nio.charset.StandardCharsets;
import java.util.Base64;

public class OnPubSub implements CloudEventsFunction {
  @Override
  public void accept(CloudEvent event) {
    if (event.getData() != null) {
      String decoded = new String(
          Base64.getDecoder().decode(new String(event.getData().toBytes(), StandardCharsets.UTF
          StandardCharsets.UTF_8);
      System.out.println("Received Pub/Sub message: " + decoded);
    }
  }
}
```

```
gcloud functions deploy on-pubsub-msg \
  --gen2 --runtime=java17 --region=asia-south1 \
  --source=. --entry-point=com.ace.functions.OnPubSub \
  --trigger-topic=fn-topic

# Test it
gcloud pubsub topics publish fn-topic --message="hello from pubsub"
```

Section 4: Google Cloud Function Hands-On — Java Spring Boot + GKE + Pub/Sub Trigger (HTTP, Cloud Storage, Pub/Sub)

Theory

Cloud Functions natively supports Java (Gen 2, via the **Functions Framework for Java**), which lets you write a function using familiar Spring-style annotations while GCP handles the HTTP server, scaling, and event delivery. Gen 2 functions run on top of **Cloud Run** under the hood, which is why they support concurrency, longer request timeouts, and larger instance sizes than Gen 1. The same Java jar can also be containerized and run as a normal GKE Deployment if you need full control over the runtime (sidergars, custom base images, service mesh, etc.) instead of the abstracted Functions Framework.

Three trigger types are commonly tested on the ACE exam: - **HTTP trigger** — synchronous request/response, good for APIs and webhooks. - **Cloud Storage trigger** — fires on `finalize`, `delete`, `archive`, or `metadataUpdate` events for a bucket. - **Pub/Sub trigger** — fires when a message is published to a topic; ideal for decoupled, asynchronous processing (e.g., image resize after upload, order processing after checkout).

Hands-On Lab

Step 1 — Scaffold a Java Functions Framework project

```
mkdir java-fn && cd java-fn
```

`pom.xml` (key dependency):

```
<dependency>
  <groupId>com.google.cloud.functions</groupId>
  <artifactId>functions-framework-api</artifactId>
  <version>1.1.0</version>
</dependency>
```

Step 2 — HTTP Function (Java)

```
package com.ace.functions;

import com.google.cloud.functions.HttpFunction;
import com.google.cloud.functions.HttpRequest;
import com.google.cloud.functions.HttpResponse;
import java.io.BufferedWriter;

public class HelloHttp implements HttpFunction {
  @Override
  public void service(HttpRequest request, HttpResponse response) throws Exception {
    BufferedWriter writer = response.getWriter();
    writer.write("Hello from Java Cloud Function (HTTP)!");
  }
}
```

Deploy:

```
gcloud functions deploy java-hello-http \
  --gen2 --runtime=java17 --region=asia-south1 \
  --source=. --entry-point=com.ace.functions.HelloHttp \
  --trigger-http --allow-unauthenticated \
  --memory=512MB
```

Step 3 — Pub/Sub-Triggered Function (Java)

```
package com.ace.functions;

import com.google.cloud.functions.CloudEventsFunction;
import io.cloudevents.CloudEvent;

public class PubSubHandler implements CloudEventsFunction {
  @Override
  public void accept(CloudEvent event) {
    System.out.println("Received Pub/Sub CloudEvent: " + event.getId());
  }
}
```

Deploy:

```
gcloud pubsub topics create java-fn-topic

gcloud functions deploy java-pubsub-fn \
  --gen2 --runtime=java17 --region=asia-south1 \
  --source=. --entry-point=com.ace.functions.PubSubHandler \
  --trigger-topic=java-fn-topic \
  --memory=512MB
```

Step 4 — Cloud Storage-Triggered Function (Java)

```
package com.ace.functions;

import com.google.cloud.functions.CloudEventsFunction;
import io.cloudevents.CloudEvent;

public class StorageHandler implements CloudEventsFunction {
  @Override
  public void accept(CloudEvent event) {
    System.out.println("Storage event type: " + event.getType() + " id: " + event.getId());
  }
}
```

Deploy:

```
gcloud storage buckets create gs://YOUR_PROJECT_ID-java-fn-bucket --location=asia-south1

gcloud functions deploy java-storage-fn \
  --gen2 --runtime=java17 --region=asia-south1 \
  --source=. --entry-point=com.ace.functions.StorageHandler \
  --trigger-event-filters="type=google.cloud.storage.object.v1.finalized" \
  --trigger-event-filters="bucket=YOUR_PROJECT_ID-java-fn-bucket"
```

Step 5 — Same logic as a GKE Deployment (Spring Boot + Pub/Sub)

If you'd rather run this as a long-lived service on GKE (instead of a Function), package the same Pub/Sub consumer logic inside a Spring Boot app using `spring-cloud-gcp-starter-pubsub`, containerize it, push to Artifact Registry, and deploy exactly as in **Section 2**:

```
docker build -t spring-pubsub-consumer:v1 .
docker tag spring-pubsub-consumer:v1 asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/spring-
docker push asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/spring-pubsub-consumer:v1

kubectl create deployment spring-pubsub-consumer \
  --image=asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/spring-pubsub-consumer:v1
```

This pattern (Functions for lightweight event glue, GKE for stateful/long-running consumers) is a common exam scenario for choosing the right compute option.

Section 5: Deploying and Implementing Data Solutions

Theory

GCP offers a data product for nearly every shape of data:

Product	Type	Use Case
Cloud SQL	Managed relational (MySQL/Postgres/SQL Server)	Traditional OLTP apps
Cloud Datastore / Firestore	Managed NoSQL document store	Mobile/web app backends
BigQuery	Serverless data warehouse	Analytics, SQL over petabytes
Cloud Spanner	Globally distributed relational	Horizontally scalable OLTP with strong consistency
Cloud Pub/Sub	Messaging	Decoupling & streaming ingestion
Cloud Bigtable	Wide-column NoSQL	High-throughput time-series/IoT
Cloud Dataproc	Managed Hadoop/Spark	Lift-and-shift big-data jobs
Cloud Dataflow	Managed Apache Beam	Unified batch + stream ETL
Cloud Storage	Object storage	Data lake, staging, backups

Loading data into these systems generally falls into one of these patterns: command-line upload (`gcloud storage cp`, `bq load`), API/SDK transfer, import/export jobs (`gcloud sql import/export`, `gcloud spanner databases import`), or streaming ingestion via **Pub/Sub → Dataflow → BigQuery/Bigtable**.

Hands-On Lab

A. Cloud SQL — Initialize + Load Data

```
gcloud sql instances create ace-sql-instance \
  --database-version=MYSQL_8_0 --tier=db-f1-micro --region=asia-south1

gcloud sql databases create ace_db --instance=ace-sql-instance

# Load data from a CSV in Cloud Storage
gcloud storage cp employees.csv gs://YOUR_PROJECT_ID-fn-bucket/
gcloud sql import csv ace-sql-instance gs://YOUR_PROJECT_ID-fn-bucket/employees.csv \
  --database=ace_db --table=employees
```

B. BigQuery — Initialize + Load Data

```
bq mk --dataset --location=asia-south1 YOUR_PROJECT_ID:ace_dataset

bq load --autodetect --source_format=CSV \
  ace_dataset.employees gs://YOUR_PROJECT_ID-fn-bucket/employees.csv
```

C. Cloud Spanner — Initialize

```
gcloud spanner instances create ace-spanner-instance \
  --config=regional-asia-south1 --description="ACE Demo" --nodes=1

gcloud spanner databases create ace-db --instance=ace-spanner-instance
```

D. Cloud Pub/Sub — Streaming Ingestion Setup

```
gcloud pubsub topics create ace-stream-topic
gcloud pubsub subscriptions create ace-stream-sub --topic=ace-stream-topic

# Publish a test message to simulate streaming data
gcloud pubsub topics publish ace-stream-topic --message='{"event":"click","user":"u1"}'
```

E. Cloud Bigtable — Initialize

```
gcloud bigtable instances create ace-bt-instance \
  --cluster=ace-bt-cluster --cluster-zone=asia-south1-a \
  --display-name="ACE Bigtable" --cluster-num-nodes=1
```

Section 6: Create Bucket & Upload (Image) Object — Java Spring Boot + GKE

Theory

Cloud Storage buckets store objects (immutable blobs) organized under a flat namespace with a globally unique bucket name. Three concepts are frequently tested:

- **Object Lifecycle Management**: Rule-based automation to transition objects between storage classes (Standard → Nearline → Coldline → Archive) or delete them, based on conditions like age, number of newer versions, or creation date. This reduces storage cost automatically without manual intervention.
- **Access Control**: Two models — **Uniform** (IAM-only, bucket-level, recommended) and **Fine-grained** (legacy ACLs, per-object). IAM roles like `roles/storage.objectViewer` or `roles/storage.objectAdmin` control who can read/write.
- **Signed URLs**: A time-limited URL granting temporary access (read or write) to a specific object without requiring the requester to have a Google account or IAM permissions — commonly used for direct browser uploads/downloads.

Hands-On Lab

Step 1 — Create the bucket

```
gcloud storage buckets create gs://YOUR_PROJECT_ID-images \
  --location=asia-south1 --uniform-bucket-level-access
```

Step 2 — Upload an image object

```
gcloud storage cp profile-pic.jpg gs://YOUR_PROJECT_ID-images/uploads/profile-pic.jpg
```

Step 3 (a) — Set an Object Lifecycle Policy

```
cat > lifecycle.json << 'EOF'
{
  "rule": [
    {
      "action": {"type": "SetStorageClass", "storageClass": "NEARLINE"},
      "condition": {"age": 30}
    },
    {
      "action": {"type": "Delete"},
      "condition": {"age": 365}
    }
  ]
}
EOF
gcloud storage buckets update gs://YOUR_PROJECT_ID-images --lifecycle-file=lifecycle.json
```

Step 3 (b) — Access Control (IAM)

```
# Grant a specific user read-only access to objects in the bucket
gcloud storage buckets add-iam-policy-binding gs://YOUR_PROJECT_ID-images \
  --member="user:[email protected]" \
  --role="roles/storage.objectViewer"
```

Step 3 (c) — Generate a Signed URL

```
gcloud storage sign-url gs://YOUR_PROJECT_ID-images/uploads/profile-pic.jpg \
  --private-key-file=service-account-key.json \
  --duration=15m
```

This returns a time-boxed HTTPS URL that expires in 15 minutes — share it directly with a client app for a temporary download link.

Step 4 — Java Spring Boot Save & Download Endpoints (with Security Features) — Full Practical Walkthrough

This walkthrough gives you a **complete, runnable** mini-project (not just fragments) covering: Maven setup → a full save/download controller with security checks → a security filter you can actually test with `curl` → containerize → deploy to GKE → verify end-to-end. Practices covered: authenticated access only, upload validation (type/size), safe server-generated object names (no path traversal), per-object authorization on download, and short-lived Signed URLs instead of public objects.

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>com.google.cloud</groupId>
    <artifactId>google-cloud-storage</artifactId>
    <version>2.36.1</version>
  </dependency>
</dependencies>
```

4.1 — Project setup (`pom.xml` key dependencies)

```
# src/main/resources/application.properties
server.port=8080
app.bucket-name=YOUR_PROJECT_ID-images
# A demo API key for this lab — in real deployments, inject this via a K8s Secret, never hardco
app.api-key=${API_KEY:demo-secret-key-123}
```

4.2 — Application properties

4.3 — A practical, testable security filter (API-key based)

Full OAuth2/JWT is the production-grade option (as shown in the config below), but it needs an external identity provider to test end-to-end. This lightweight filter enforces the **same principle — no anonymous access — and is directly testable with `curl`.**

```
@Component
public class ApiKeyAuthFilter extends OncePerRequestFilter {

  @Value("${app.api-key}")
  private String expectedApiKey;

  @Override
  protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                  FilterChain chain) throws ServletException, IOException {
    String providedKey = request.getHeader("X-API-Key");

    if (providedKey == null || !providedKey.equals(expectedApiKey)) {
      response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
      response.getWriter().write("Missing or invalid X-API-Key header.");
      return;
    }

    // Mark the request as authenticated so @PreAuthorize("isAuthenticated()") passes downstrea
    Authentication auth = new UsernamePasswordAuthenticationToken("api-client", null, List.of()
    SecurityContextHolder.getContext().setAuthentication(auth);
    chain.doFilter(request, response);
  }
}
```

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

  private final ApiKeyAuthFilter apiKeyAuthFilter;

  public SecurityConfig(ApiKeyAuthFilter apiKeyAuthFilter) {
    this.apiKeyAuthFilter = apiKeyAuthFilter;
  }

  @Bean
  public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
      .csrf(csrf -> csrf.disable())
      .authorizeHttpRequests(auth -> auth
          .requestMatchers("/actuator/health").permitAll()
          .anyRequest().authenticated())
      .addFilterBefore(apiKeyAuthFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
  }

  // ---- Production alternative: swap the API-key filter for a real OAuth2/JWT resource server
  // .oauth2ResourceServer(oauth2 -> oauth2.jwt())  // validates a Google-signed ID token inste
```

```
}
```

```
@Service
public class OwnerService {
  // objectName -> owning user. In production this is a row in Cloud SQL/Firestore, not memory.
  private final Map<String, String> ownership = new ConcurrentHashMap<>();

  public void recordOwner(String objectName, String owner) {
    ownership.put(objectName, owner);
  }

  public boolean canRead(String requester, String objectName) {
    // For this lab, any authenticated caller may read any object they (or the demo client) upl
    // Tighten this to `requester.equals(ownership.get(objectName))` once you add real user ide
    return ownership.containsKey(objectName);
  }
}
```

4.4 — A simple ownership record (in-memory, swap for a real DB table in production)

```
@RestController
@RequestMapping("/files")
public class FileStorageController {

  @Value("${app.bucket-name}")
  private String bucket;

  private static final Set<String> ALLOWED_TYPES = Set.of("image/jpeg", "image/png", "image/web
  private static final long MAX_FILE_SIZE_BYTES = 5 * 1024 * 1024; // 5 MB

  private final Storage storage = StorageOptions.getDefaultInstance().getService();
  private final OwnerService ownerService;

  public FileStorageController(OwnerService ownerService) {
    this.ownerService = ownerService;
  }

  // ---------- SAVE (Upload) ----------
  @PostMapping("/upload")
  @PreAuthorize("isAuthenticated()")
  public ResponseEntity<String> upload(@RequestParam("file") MultipartFile file) throws IOExcep

    if (file.isEmpty() || !ALLOWED_TYPES.contains(file.getContentType())) {
      return ResponseEntity.badRequest().body("Only JPEG/PNG/WEBP images are allowed.");
    }
    if (file.getSize() > MAX_FILE_SIZE_BYTES) {
      return ResponseEntity.badRequest().body("File exceeds 5MB limit.");
    }

    // Never trust the client-supplied filename — generate a safe, random object name.
    String safeName = "uploads/" + UUID.randomUUID() + sanitizeExtension(file.getOriginalFilena
```

```
    BlobId blobId = BlobId.of(bucket, safeName);
    BlobInfo blobInfo = BlobInfo.newBuilder(blobId).setContentType(file.getContentType()).build

    // doesNotExist() => fail if an object with this exact name already exists (no silent overw
    storage.create(blobInfo, file.getBytes(), Storage.BlobTargetOption.doesNotExist());

    ownerService.recordOwner(safeName, "api-client");
    return ResponseEntity.ok("Uploaded as: " + safeName);
  }

  // ---------- DOWNLOAD (direct, authenticated & authorized) ----------
  @GetMapping("/download/**")
  @PreAuthorize("isAuthenticated()")
  public ResponseEntity<byte[]> download(HttpServletRequest request, Authentication auth) {
    String objectName = extractObjectName(request, "/files/download/");

    if (!ownerService.canRead(auth.getName(), objectName)) {
      return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
    }

    Blob blob = storage.get(BlobId.of(bucket, objectName));
    if (blob == null) {
      return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok()
        .contentType(MediaType.parseMediaType(blob.getContentType()))
        .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=\"" + objectName + "\"")
        .body(blob.getContent());
  }

  // ---------- DOWNLOAD (indirect, via short-lived Signed URL) ----------
  @GetMapping("/download-url/**")
  @PreAuthorize("isAuthenticated()")
  public ResponseEntity<String> downloadUrl(HttpServletRequest request, Authentication auth) {
    String objectName = extractObjectName(request, "/files/download-url/");

    if (!ownerService.canRead(auth.getName(), objectName)) {
      return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
    }

    BlobInfo blobInfo = BlobInfo.newBuilder(bucket, objectName).build();
    URL signedUrl = storage.signUrl(blobInfo, 15, TimeUnit.MINUTES, Storage.SignUrlOption.withV

    return ResponseEntity.ok(signedUrl.toString()); // client downloads directly from GCS; URL
  }

  private String extractObjectName(HttpServletRequest request, String prefix) {
    String path = request.getRequestURI().substring(request.getContextPath().length());
    String rawName = path.substring(path.indexOf(prefix) + prefix.length());
    return "uploads/" + rawName.replaceAll("[^a-zA-Z0-9._/-]", "").replace("../", "");
  }
```

```
  private String sanitizeExtension(String originalFilename) {
    String ext = StringUtils.getFilenameExtension(originalFilename);
    return (ext != null && ext.matches("[a-zA-Z0-9]{1,5}")) ? "." + ext.toLowerCase() : "";
  }
}
```

4.5 — The full controller (save + both download styles) Before containerizing this app, note how it authenticates to Cloud Storage: the `Storage` client uses **Application Default Credentials**, resolved automatically from the environment it runs in. On GKE that means the Pod's Kubernetes Service Account, bound via **Workload Identity** to a GCP service account holding `roles/storage.objectAdmin` **scoped to this one bucket only** (never project-wide, and never a downloaded JSON key baked into the image). That's what lets `storage.create()` and `storage.get()` succeed in the controller above without any credentials file ever touching the container — the identity comes from the platform, not the code.

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -q clean package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

4.6 — Dockerfile (multi-stage, same pattern as Section 2)

```
docker build -t file-storage-app:v1 .
docker tag file-storage-app:v1 asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/file-storage-
docker push asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/file-storage-app:v1

kubectl create deployment file-storage-app \
  --image=asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/file-storage-app:v1

kubectl set env deployment/file-storage-app API_KEY=demo-secret-key-123
kubectl expose deployment file-storage-app --type=LoadBalancer --port=80 --target-port=8080
kubectl get service file-storage-app
```

4.7 — Build, push, and deploy to GKE In production, put `API_KEY` in a **Kubernetes Secret** (`kubectl create secret generic api-key --from-literal=API_KEY=...`) and reference it via `valueFrom.secretKeyRef` instead of `kubectl set env`. Also bind the Pod's Kubernetes Service Account to a GCP service account with `roles/storage.objectAdmin` **scoped to this one bucket only** via Workload Identity, so the app authenticates to GCS without any embedded key.

```
EXTERNAL_IP=$(kubectl get service file-storage-app -o jsonpath='{.status.loadBalancer.ingress[0

# 1) No API key -> rejected
curl -i http://$EXTERNAL_IP/files/upload -F "file=@profile-pic.jpg"
```

```
# HTTP/1.1 401 ... Missing or invalid X-API-Key header.

# 2) With API key -> upload succeeds, returns the server-generated object name
curl -i -H "X-API-Key: demo-secret-key-123" \
  http://$EXTERNAL_IP/files/upload -F "file=@profile-pic.jpg"
# 200 OK  ->  Uploaded as: uploads/6c1f2e4a-....jpg

# 3) Direct authenticated download of that object
curl -i -H "X-API-Key: demo-secret-key-123" \
  http://$EXTERNAL_IP/files/download/6c1f2e4a-....jpg -o downloaded.jpg

# 4) Get a 15-minute Signed URL instead, then download with plain curl (no API key needed on th
curl -H "X-API-Key: demo-secret-key-123" \
  http://$EXTERNAL_IP/files/download-url/6c1f2e4a-....jpg
curl -o downloaded2.jpg "<signed-url-returned-above>"
```

4.8 — Test it end-to-end with `curl` This confirms all three security properties in practice: **unauthenticated calls are rejected**, **uploads are validated and safely named**, and **downloads are gated per object** whether served directly or via a time-boxed Signed URL.

Section 7: Labs — Cloud Spanner, Cloud Memorystore, Cloud Firestore, Cloud Bigtable

Theory

- **Cloud Spanner**: A globally distributed, horizontally scalable relational database with external strong consistency (via TrueTime). Combines SQL + ACID transactions with NoSQL-like horizontal scale — used where you need both.
- **Cloud Memorystore**: Fully managed **Redis** or **Memcached**, used for caching, session storage, and leaderboards — sits in front of a primary datastore to reduce read latency.
- **Cloud Firestore**: Serverless NoSQL document database (successor to Datastore Mode/Native Mode unification), with real-time sync — ideal for mobile/web app backends.
- **Cloud Bigtable**: Wide-column NoSQL store built for very high throughput and low latency at petabyte scale — used for time-series, IoT telemetry, and ad-tech.

Hands-On Lab

A. Cloud Spanner

```
gcloud spanner instances create spanner-lab --config=regional-asia-south1 --nodes=1 --descripti
gcloud spanner databases create lab-db --instance=spanner-lab --ddl="CREATE TABLE Singers (Sing
gcloud spanner rows insert --database=lab-db --instance=spanner-lab --table=Singers --data=Sing
gcloud spanner databases execute-sql lab-db --instance=spanner-lab --sql="SELECT * FROM Singers
```

B. Cloud Memorystore (Redis)

```
gcloud redis instances create memorystore-lab \
  --size=1 --region=asia-south1 --redis-version=redis_7_0

gcloud redis instances describe memorystore-lab --region=asia-south1 --format="value(host,port)
# Connect from a Compute Engine VM in the same VPC using redis-cli <HOST> <PORT>
```

C. Cloud Firestore

```
gcloud firestore databases create --location=asia-south1 --type=firestore-native
```

Add a document via the SDK using a **Java Spring Boot** service:

```
@Service
public class FirestoreUserService {

  private final Firestore db = FirestoreOptions.getDefaultInstance().getService();

  public void saveAndReadUser() throws Exception {
    Map<String, Object> user = new HashMap<>();
    user.put("name", "Marc");
    user.put("age", 30);

    db.collection("users").document("u1").set(user).get();

    DocumentSnapshot snapshot = db.collection("users").document("u1").get().get();
```

```
    System.out.println("Fetched user: " + snapshot.getData());
  }
}
```

Add `com.google.cloud:google-cloud-firestore` to `pom.xml`, then containerize and deploy this Spring Boot service to GKE using the same build → push → `kubectl create deployment` flow as **Section 2**, granting the Pod's service account `roles/datastore.user` on the project via Workload Identity.

D. Cloud Bigtable

```
gcloud bigtable instances create bigtable-lab \
  --cluster=bigtable-lab-cluster --cluster-zone=asia-south1-a \
  --display-name="Bigtable Lab" --cluster-num-nodes=1

# Use cbt CLI to create a table and write data
gcloud components install cbt
cbt -instance=bigtable-lab createtable events
cbt -instance=bigtable-lab createfamily events cf1
cbt -instance=bigtable-lab set events row1 cf1:col1=value1
cbt -instance=bigtable-lab read events
```

Section 8: Cloud Dataflow

Theory

Cloud Dataflow is Google's fully managed, serverless service for executing **Apache Beam** pipelines. Apache Beam provides one unified programming model (in Java, Python, or Go) for both **batch** and **streaming** data processing — you write the pipeline once, and choose the runner/execution mode. Dataflow auto-scales workers, handles shuffling, and offers many **pre-built templates** (e.g., "Pub/Sub to BigQuery", "GCS Text to BigQuery") so you often don't need to write custom code at all — but you can supply a **custom template** (Flex Template) when built-ins don't fit.

Key benefits over self-managed Spark/Flink: no cluster management, autoscaling, and unified batch/stream semantics (windowing, triggers, watermarks) for handling late/out-of-order data.

Hands-On Lab

A. Run a pre-built template: Pub/Sub → BigQuery

```
bq mk --dataset YOUR_PROJECT_ID:dataflow_demo
bq mk --table YOUR_PROJECT_ID:dataflow_demo.events message:STRING,publish_time:TIMESTAMP

gcloud pubsub topics create dataflow-input-topic

gcloud dataflow jobs run pubsub-to-bq-job \
  --gcs-location=gs://dataflow-templates-asia-south1/latest/PubSub_Subscription_to_BigQuery \
  --region=asia-south1 \
  --staging-location=gs://YOUR_PROJECT_ID-fn-bucket/staging \
  --parameters inputSubscription=projects/YOUR_PROJECT_ID/subscriptions/dataflow-input-sub,outp
```

B. Run a custom Apache Beam pipeline (Java, Spring Boot-packaged)

```
public class ActiveEmployeesPipeline {

  public static void main(String[] args) {
    DataflowPipelineOptions options = PipelineOptionsFactory
        .fromArgs(args).withValidation().as(DataflowPipelineOptions.class);
    options.setRunner(DataflowRunner.class);
    options.setProject("YOUR_PROJECT_ID");
    options.setRegion("asia-south1");
    options.setTempLocation("gs://YOUR_PROJECT_ID-fn-bucket/temp");

    Pipeline p = Pipeline.create(options);

    p.apply("ReadFromGCS", TextIO.read().from("gs://YOUR_PROJECT_ID-fn-bucket/employees.csv"))
     .apply("ParseLine", MapElements.into(TypeDescriptors.strings())
         .via((String line) -> line))
     .apply("FilterActive", Filter.by((String line) -> line.split(",")[2].equals("active")))
     .apply("WriteOut", TextIO.write().to("gs://YOUR_PROJECT_ID-fn-bucket/output/active_employe

    p.run();
```

```
  }
}
```

Add `org.apache.beam:beam-sdks-java-core` and `org.apache.beam:beam-runners-google-cloud-dataflow-java` to `pom.xml`. This class can live inside the same Spring Boot project used elsewhere in this guide (e.g., invoked from a `CommandLineRunner` bean or a REST endpoint that kicks off the pipeline on demand) — build and run it with Maven:

```
mvn compile exec:java \
  -Dexec.mainClass=com.ace.dataflow.ActiveEmployeesPipeline \
  -Dexec.args="--runner=DataflowRunner --project=YOUR_PROJECT_ID --region=asia-south1 --tempLoc
```

C. Monitor the job

```
gcloud dataflow jobs list --region=asia-south1
gcloud dataflow jobs describe JOB_ID --region=asia-south1
```

Or **Console → Dataflow → Jobs → select job** to view the visual pipeline DAG, autoscaling graph, and per-stage throughput.

Section 9: Cloud Dataproc

Theory

Cloud Dataproc is a managed **Hadoop and Spark** service — use it when you already have Hadoop/Spark/Hive/Pig jobs and want to "lift and shift" them to GCP without rewriting to Beam. Dataproc clusters spin up in ~90 seconds, can autoscale, and support **ephemeral clusters** (create → run job → delete) to minimize cost, since you only pay for the cluster while it's running.

Hands-On Lab — Demo: Create a Cluster and Submit a Spark Job to Calculate π

Step 1 — Enable API and create a Dataproc cluster

```
gcloud services enable dataproc.googleapis.com

gcloud dataproc clusters create ace-dataproc-cluster \
  --region=asia-south1 \
  --zone=asia-south1-a \
  --master-machine-type=n1-standard-2 \
  --worker-machine-type=n1-standard-2 \
  --num-workers=2
```

Step 2 — Submit the built-in Spark Pi job

```
gcloud dataproc jobs submit spark \
  --cluster=ace-dataproc-cluster \
  --region=asia-south1 \
  --class=org.apache.spark.examples.SparkPi \
  --jars=file:///usr/lib/spark/examples/jars/spark-examples.jar \
  -- 1000
```

The trailing `1000` is the number of Monte Carlo sample partitions — higher means a more precise π estimate at the cost of more compute time.

Step 3 — Check job status and output

```
gcloud dataproc jobs list --cluster=ace-dataproc-cluster --region=asia-south1
gcloud dataproc jobs wait JOB_ID --region=asia-south1
```

The driver output log will contain a line like `Pi is roughly 3.14159...`.

Step 4 — Submit a Hadoop MapReduce job (word count) for comparison

```
gcloud dataproc jobs submit hadoop \
  --cluster=ace-dataproc-cluster --region=asia-south1 \
  --jar=file:///usr/lib/hadoop-mapreduce/hadoop-mapreduce-examples.jar \
  -- wordcount gs://YOUR_PROJECT_ID-fn-bucket/employees.csv gs://YOUR_PROJECT_ID-fn-bucket/outp
```

Step 5 — Cleanup (ephemeral cluster pattern)

```
gcloud dataproc clusters delete ace-dataproc-cluster --region=asia-south1
```

In production, wrap steps 1–5 into a single script or Cloud Composer DAG so the cluster only exists for the duration of the job — this is the standard ACE-exam pattern for cost-efficient batch processing.

Section 10: Cloud Pub/Sub (Hands-On)

Theory

Cloud Pub/Sub is a fully managed, asynchronous messaging service that decouples publishers from subscribers and scales to millions/billions of messages per day. Core concepts:

- **Topic**: A named resource publishers send messages to.
- **Subscription**: A named resource representing the stream of messages from a topic to a specific subscriber.
- **Push vs Pull**: In **Pull**, the subscriber actively requests messages (good for batch workers). In **Push**, Pub/Sub actively delivers messages via HTTP POST to a configured endpoint (good for serverless consumers like Cloud Functions/Cloud Run).
- **At-least-once delivery**: Consumers must be idempotent since a message can be redelivered.
- **Ack deadline**: Time the subscriber has to acknowledge a message before Pub/Sub redelivers it.

Hands-On Lab

Step 1 — Create a topic and subscriptions (Pull and Push)

```
gcloud pubsub topics create orders-topic

# Pull subscription
gcloud pubsub subscriptions create orders-pull-sub --topic=orders-topic

# Push subscription (delivers to an HTTPS endpoint, e.g. a Cloud Run service)
gcloud pubsub subscriptions create orders-push-sub --topic=orders-topic \
  --push-endpoint=https://hello-run-xyz.a.run.app/pubsub-handler
```

Step 2 — Publish messages

```
gcloud pubsub topics publish orders-topic --message='{"orderId":101,"status":"created"}'
gcloud pubsub topics publish orders-topic --message='{"orderId":102,"status":"created"}' --attr
```

Step 3 — Pull messages (synchronous pull)

```
gcloud pubsub subscriptions pull orders-pull-sub --auto-ack --limit=10
```

Step 4 — Verify push delivery

Check your Cloud Run/Cloud Function logs — Pub/Sub will have POSTed the message body (base64-encoded) automatically once the push subscription was created; no polling code needed on the consumer side.

Step 5 — Scale test (simulate billions/day pattern)

```
for i in {1..100}; do
  gcloud pubsub topics publish orders-topic --message="{\"orderId\":$i}"
done
```

Pub/Sub requires no capacity planning for this — it auto-scales transparently; you only manage topics, subscriptions, and IAM.

Step 6 — Monitor backlog

```
gcloud pubsub subscriptions describe orders-pull-sub \
  --format="value(name)"
```

Or **Console → Pub/Sub → Subscriptions → orders-pull-sub → Metrics** to view "Unacked message count" — a growing backlog signals subscribers can't keep up and need more instances/workers.

Section 11: Deploying and Implementing Networking Resources

Theory

- **VPC (Virtual Private Cloud)**: A global, private network. **Custom-mode** VPCs let you define subnets per region explicitly (recommended over auto-mode for production). A **Shared VPC** lets a "host" project share its network with "service" projects, centralizing network administration while letting app teams manage their own resources.
- **Compute Engine networking options**: Instances can have an internal-only IP (no internet access unless via NAT), **Private Google Access** (reach Google APIs without a public IP), static external/internal IPs (persist across stop/start), and **network tags** (used to target firewall rules).
- **Firewall rules**: Stateful, apply at the VPC level, and are evaluated by priority. Rules can target by IP range, network tag, or service account, and apply to **ingress** (incoming) or **egress** (outgoing) traffic.
- **Cloud VPN**: Site-to-site IPsec VPN connecting your on-prem network to a GCP VPC over the public internet, encrypted.
- **Load Balancers**: Choose based on traffic type/scope —
  - **Global HTTP(S) LB**: Layer 7, global, for web traffic (supports URL maps, SSL offload, CDN).
  - **Global SSL Proxy LB**: Layer 4 (TCP+SSL termination), global, non-HTTP(S) TCP traffic needing SSL offload.
  - **Global TCP Proxy LB**: Layer 4, global, plain TCP (no SSL termination).
  - **Regional Network LB**: Layer 4, regional, pass-through (preserves client IP), for TCP/UDP.
  - **Regional Internal LB**: Layer 4, regional, private (internal-only) load balancing between internal clients and backends.

Hands-On Lab

A. Create a Custom-Mode VPC with Subnets

```
gcloud compute networks create ace-vpc --subnet-mode=custom

gcloud compute networks subnets create ace-subnet-1 \
  --network=ace-vpc --range=10.10.1.0/24 --region=asia-south1 \
  --enable-private-ip-google-access
```

B. Launch a Compute Engine instance with custom networking

```
gcloud compute instances create ace-vm \
  --zone=asia-south1-a \
  --network=ace-vpc --subnet=ace-subnet-1 \
  --no-address \
  --tags=web-server \
  --machine-type=e2-medium
```

`--no-address` gives it an internal-only IP; `--enable-private-ip-google-access` (set on the subnet) lets it reach Google APIs (e.g., Cloud Storage) without a public IP.

C. Ingress/Egress Firewall Rules

```
# Ingress: allow HTTP from anywhere to instances tagged web-server
gcloud compute firewall-rules create allow-http-ingress \
  --network=ace-vpc --direction=INGRESS --action=ALLOW \
  --rules=tcp:80 --source-ranges=0.0.0.0/0 --target-tags=web-server

# Egress: deny all outbound except to a specific IP range (e.g., internal API)
gcloud compute firewall-rules create restrict-egress \
  --network=ace-vpc --direction=EGRESS --action=DENY \
  --rules=all --destination-ranges=0.0.0.0/0 --priority=1000

gcloud compute firewall-rules create allow-egress-internal-api \
  --network=ace-vpc --direction=EGRESS --action=ALLOW \
  --rules=tcp:443 --destination-ranges=10.20.0.0/16 --priority=900
```

D. Cloud VPN — Site-to-Site to an External Network

```
gcloud compute addresses create vpn-static-ip --region=asia-south1

gcloud compute vpn-gateways create ace-vpn-gw \
  --network=ace-vpc --region=asia-south1

gcloud compute vpn-tunnels create ace-vpn-tunnel \
  --region=asia-south1 --peer-address=203.0.113.5 \
  --ike-version=2 --shared-secret=YOUR_PSK \
  --target-vpn-gateway=ace-vpn-gw \
  --local-traffic-selector=10.10.1.0/24 \
  --remote-traffic-selector=192.168.1.0/24
```

E. Global HTTP(S) Load Balancer

```
gcloud compute instance-groups managed create ace-mig \
  --zone=asia-south1-a --size=2 --template=ace-instance-template

gcloud compute backend-services create ace-backend-service \
  --protocol=HTTP --port-name=http --global \
  --health-checks=ace-http-health-check

gcloud compute backend-services add-backend ace-backend-service \
  --instance-group=ace-mig --instance-group-zone=asia-south1-a --global

gcloud compute url-maps create ace-url-map --default-service=ace-backend-service

gcloud compute target-http-proxies create ace-http-proxy --url-map=ace-url-map

gcloud compute forwarding-rules create ace-http-lb-rule \
  --global --target-http-proxy=ace-http-proxy --ports=80
```

Section 12: Organization with Project1 and Project2 — Host Project using Shared VPC and VPC Peering (Hands-On)

Theory

- **Shared VPC**: One project (the **Host Project**) owns the VPC network. Other projects (**Service Projects**) are attached to it and their resources (VMs, GKE clusters) can use the host's subnets directly — as if they were in the same network — while IAM/billing/resources remain separately managed per service project. This is the recommended pattern for centralizing network control across teams in an Organization.
- **VPC Peering**: Connects **two separate VPC networks** (which can be in different projects/orgs) so resources can communicate using internal IPs, without a Shared VPC relationship, VPN, or public IPs. Unlike Shared VPC, peered networks remain administratively independent — each side manages its own firewall rules and IAM. Peering is **not transitive** (if A peers with B, and B peers with C, A cannot reach C through B).

Choose **Shared VPC** for centralized network governance across many projects in one org; choose **VPC Peering** for connecting a small number of independently-administered networks (including across different organizations).

Hands-On Lab

Part A — Shared VPC (Host: Project1, Service: Project2)

Step 1 — Enable Shared VPC on the Host Project (Project1)

```
gcloud compute shared-vpc enable PROJECT1_ID
```

Step 2 — Attach Project2 as a Service Project

```
gcloud compute shared-vpc associated-projects add PROJECT2_ID \
  --host-project=PROJECT1_ID
```

Step 3 — Create the shared subnet in Project1

```
gcloud compute networks create shared-vpc-net --project=PROJECT1_ID --subnet-mode=custom

gcloud compute networks subnets create shared-subnet-1 \
  --project=PROJECT1_ID --network=shared-vpc-net \
  --range=10.30.1.0/24 --region=asia-south1
```

Step 4 — Grant the Project2 service account/user access to the subnet

```
gcloud projects add-iam-policy-binding PROJECT1_ID \
  --member="serviceAccount:PROJECT2_NUMBER@[email protected]" \
  --role="roles/compute.networkUser" \
  --condition=None
```

Step 5 — Launch a VM in Project2 using Project1's shared subnet

```
gcloud compute instances create service-project-vm \
  --project=PROJECT2_ID --zone=asia-south1-a \
  --network=projects/PROJECT1_ID/global/networks/shared-vpc-net \
  --subnet=projects/PROJECT1_ID/regions/asia-south1/subnetworks/shared-subnet-1
```

Part B — VPC Peering (Project1 VPC ↔ Project2 VPC)

Step 1 — Create two independent VPCs (one per project)

```
gcloud compute networks create vpc-a --project=PROJECT1_ID --subnet-mode=custom
gcloud compute networks subnets create vpc-a-subnet --project=PROJECT1_ID \
  --network=vpc-a --range=10.40.1.0/24 --region=asia-south1

gcloud compute networks create vpc-b --project=PROJECT2_ID --subnet-mode=custom
gcloud compute networks subnets create vpc-b-subnet --project=PROJECT2_ID \
  --network=vpc-b --range=10.50.1.0/24 --region=asia-south1
```

Step 2 — Peer from vpc-a to vpc-b

```
gcloud compute networks peerings create peer-a-to-b \
  --project=PROJECT1_ID --network=vpc-a \
  --peer-project=PROJECT2_ID --peer-network=vpc-b
```

Step 3 — Peer back from vpc-b to vpc-a (peering must be established both ways)

```
gcloud compute networks peerings create peer-b-to-a \
  --project=PROJECT2_ID --network=vpc-b \
  --peer-project=PROJECT1_ID --peer-network=vpc-a
```

Step 4 — Verify peering status

```
gcloud compute networks peerings list --project=PROJECT1_ID --network=vpc-a
```

Status should show `ACTIVE` on both sides once complete. VMs in `vpc-a-subnet` and `vpc-b-subnet` can now reach each other via internal IPs (subject to firewall rules on each side).

Section 13: Managing Google Kubernetes Engine Resources

Theory

Day-2 operations on GKE revolve around observing and adjusting the cluster's inventory (nodes, pods, services), the container images they run, and specialized workload types like **StatefulSets** (for apps needing stable network identity and persistent storage, e.g. databases). All of this can be done via **Cloud Console**, **Cloud Shell**, or the **Cloud SDK** (`gcloud`/`kubectl`) — the exam expects familiarity with all three surfaces.

Hands-On Lab

A. View current cluster inventory

```
kubectl get nodes -o wide
kubectl get pods --all-namespaces
kubectl get services --all-namespaces
```

B. Browse container image repository

```
gcloud artifacts docker images list asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo
gcloud artifacts docker images describe asia-south1-docker.pkg.dev/YOUR_PROJECT_ID/my-repo/hell
```

Console → Artifact Registry → my-repo to browse image tags, digests, and vulnerability scan results visually.

C. Work with node pools

```
# Add a new node pool (e.g., for a different workload profile)
gcloud container node-pools create high-mem-pool \
  --cluster=ace-demo-cluster --zone=asia-south1-a \
  --machine-type=n2-highmem-2 --num-nodes=2

# Edit (resize) a node pool
gcloud container clusters resize ace-demo-cluster \
  --node-pool=high-mem-pool --num-nodes=3 --zone=asia-south1-a

# Remove a node pool
gcloud container node-pools delete high-mem-pool \
  --cluster=ace-demo-cluster --zone=asia-south1-a
```

D. Work with pods

```
kubectl run test-pod --image=nginx
kubectl edit pod test-pod          # live edit (limited fields)
kubectl delete pod test-pod
```

E. Work with services

```
kubectl expose pod test-pod --port=80 --target-port=80 --name=test-svc --type=ClusterIP
kubectl edit service test-svc
kubectl delete service test-svc
```

F. Work with stateful applications

```
# Create a StorageClass-backed PersistentVolumeClaim
cat > pvc.yaml << 'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 10Gi
EOF
kubectl apply -f pvc.yaml

# Deploy a StatefulSet (e.g., a simple database) referencing the PVC template
kubectl apply -f statefulset.yaml
kubectl get statefulsets
kubectl get pvc
```

G. Management interfaces

All of the above are equally achievable in **Cloud Console** (Kubernetes Engine section), **Cloud Shell** (pre-authenticated `kubectl`/`gcloud`), or a local **Cloud SDK** install — the ACE exam frequently tests knowing *which* interface is fastest/most appropriate for a given scenario (e.g., Cloud Shell for quick one-off debugging without local setup).

Section 14: Managing App Engine and Cloud Run Resources

Theory

Post-deployment management focuses on two levers: **traffic distribution** across versions/revisions, and **scaling parameters** that trade off cost vs. latency (cold starts). Getting these right is a common real-world (and exam) scenario: e.g., rolling out a canary release safely, or preventing cold starts for a latency-sensitive API by setting a minimum instance floor.

Hands-On Lab

A. Adjusting App Engine traffic splitting

```
# Shift traffic gradually from v1 to v2
gcloud app services set-traffic default --splits=v1=0.5,v2=0.5
gcloud app services set-traffic default --splits=v2=1.0    # full cutover

# Split by IP or cookie instead of random (for session stickiness)
gcloud app services set-traffic default --splits=v1=0.5,v2=0.5 --split-by=ip
```

B. Adjusting Cloud Run traffic splitting

```
gcloud run services update-traffic hello-run \
  --to-revisions=hello-run-00001-abc=50,hello-run-00002-xyz=50
```

C. Setting App Engine autoscaling parameters

```
# app.yaml
automatic_scaling:
  min_instances: 1
  max_instances: 10
  target_cpu_utilization: 0.6
  max_concurrent_requests: 50
```

```
gcloud app deploy app.yaml
```

D. Setting Cloud Run autoscaling parameters

```
gcloud run services update hello-run \
  --min-instances=1 --max-instances=20 \
  --concurrency=80 --cpu=1 --memory=512Mi
```

E. Management interfaces

- **Console**: App Engine → Versions (traffic slider UI); Cloud Run → Service → Revisions tab (traffic slider UI).
- **Cloud Shell / SDK**: commands above.

Section 15: Managing Storage and Database Solutions

Theory

Ongoing management of data services covers: moving/reclassifying storage objects for cost optimization, running and estimating the cost of analytical queries, and ensuring recoverability (backups) of stateful databases. **Estimating BigQuery cost before running a query** is especially important since BigQuery on-demand pricing is billed per byte scanned — a `dry_run` avoids surprise bills on large tables.

Hands-On Lab

A. Moving objects between buckets

```
gcloud storage mv gs://YOUR_PROJECT_ID-images/uploads/profile-pic.jpg \
  gs://YOUR_PROJECT_ID-images-archive/uploads/profile-pic.jpg
```

B. Converting storage classes

```
gcloud storage objects update gs://YOUR_PROJECT_ID-images/uploads/profile-pic.jpg \
  --storage-class=COLDLINE

# Or convert an entire bucket's default class
gcloud storage buckets update gs://YOUR_PROJECT_ID-images --default-storage-class=NEARLINE
```

C. Setting object lifecycle policies

(See Section 6, Step 3a — same `lifecycle.json` + `gcloud storage buckets update --lifecycle-file` pattern.)

D. Executing queries against data instances

```
# BigQuery
bq query --use_legacy_sql=false 'SELECT name, age FROM ace_dataset.employees LIMIT 10'

# Cloud SQL
gcloud sql connect ace-sql-instance --user=root
# then at the mysql> prompt: SELECT * FROM employees LIMIT 10;

# Cloud Spanner
gcloud spanner databases execute-sql lab-db --instance=spanner-lab \
  --sql="SELECT * FROM Singers"
```

E. Estimating BigQuery query cost (dry run)

```
bq query --use_legacy_sql=false --dry_run \
  'SELECT * FROM `bigquery-public-data.samples.shakespeare`'
```

This returns "This query will process X bytes" without executing it — multiply by BigQuery's per-TB pricing to estimate cost before running expensive queries.

F. Backing up and restoring data instances

```
# Cloud SQL — on-demand backup
gcloud sql backups create --instance=ace-sql-instance

# Cloud SQL — restore from a backup
gcloud sql backups restore BACKUP_ID --restore-instance=ace-sql-instance

# Cloud Datastore/Firestore — export/import
gcloud firestore export gs://YOUR_PROJECT_ID-fn-bucket/firestore-backup
gcloud firestore import gs://YOUR_PROJECT_ID-fn-bucket/firestore-backup
```

G. Reviewing job status (Dataproc, Dataflow, BigQuery)

```
gcloud dataproc jobs list --region=asia-south1
gcloud dataflow jobs list --region=asia-south1
bq ls -j --max_results=10
```

H. Management interfaces

All operations above are available in **Console** (Cloud Storage browser, BigQuery Studio, SQL/Spanner instance pages), **Cloud Shell**, and the **Cloud SDK**.

Section 16: Managing Networking Resources

Theory

Networking maintenance tasks typically involve growing capacity (adding subnets, expanding IP ranges as an org scales) and reserving static IPs so critical resources keep a predictable address across restarts/recreation.

Hands-On Lab

A. Adding a subnet to an existing VPC

```
gcloud compute networks subnets create ace-subnet-2 \
  --network=ace-vpc --range=10.10.2.0/24 --region=asia-south2
```

B. Expanding a subnet's IP range

```
gcloud compute networks subnets expand-ip-range ace-subnet-1 \
  --region=asia-south1 --prefix-length=23
```

> Note: You can only expand (increase the address space of) a subnet's primary range — you cannot shrink it, and the new range must not overlap with existing subnets.

C. Reserving static external and internal IP addresses

```
# Static external IP
gcloud compute addresses create ace-static-ext-ip --region=asia-south1

# Static internal IP
gcloud compute addresses create ace-static-int-ip \
  --region=asia-south1 --subnet=ace-subnet-1 --addresses=10.10.1.100

# Attach the static external IP to a running VM
gcloud compute instances add-access-config ace-vm \
  --zone=asia-south1-a --address=$(gcloud compute addresses describe ace-static-ext-ip --region
```

D. Management interfaces

- **Console**: VPC Network → Subnets (add/expand), VPC Network → IP addresses (reserve).
- **Cloud Shell/SDK**: commands above.

Section 17: Monitoring and Logging

Theory

Cloud Monitoring (formerly Stackdriver Monitoring) and **Cloud Logging** (formerly Stackdriver Logging) provide observability across all GCP resources:

- **Alerting policies**: Trigger notifications (email, SMS, PagerDuty, Slack, webhook) when a metric crosses a threshold (e.g., CPU > 90% for 5 minutes).
- **Custom metrics**: When built-in metrics aren't enough, you can define your own (log-based metrics, or via the Monitoring API) to track business-specific signals (e.g., "orders failed").
- **Log sinks**: Export logs matching a filter to an external destination — Cloud Storage (archival), BigQuery (analysis), or Pub/Sub (real-time processing/on-prem forwarding).
- **Logs Explorer**: Query and filter logs using Cloud Logging's query language.
- **Cloud Trace**: Distributed tracing showing per-request latency across microservices.
- **Cloud Debugger**: Live production debugging without stopping the app (deprecated for new usage as of recent GCP versions in favor of alternative tooling — verify current availability if this is critical to your exam version).

Hands-On Lab

A. Create a Cloud Monitoring alert based on resource metrics

```
cat > alert-policy.json << 'EOF'
{
  "displayName": "High CPU Alert",
  "conditions": [{
    "displayName": "CPU > 80%",
    "conditionThreshold": {
      "filter": "resource.type=\"gce_instance\" AND metric.type=\"compute.googleapis.com/instan
      "comparison": "COMPARISON_GT",
      "thresholdValue": 0.8,
      "duration": "300s"
    }
  }],
  "notificationChannels": []
}
EOF
gcloud alpha monitoring policies create --policy-from-file=alert-policy.json
```

B. Create a custom (log-based) metric

```
gcloud logging metrics create failed_orders_count \
  --description="Counts failed order log entries" \
  --log-filter='resource.type="cloud_run_revision" AND jsonPayload.status="FAILED"'
```

C. Configure a log sink (export to BigQuery)

```
bq mk --dataset YOUR_PROJECT_ID:log_export_dataset

gcloud logging sinks create ace-log-sink \
```

```
  bigquery.googleapis.com/projects/YOUR_PROJECT_ID/datasets/log_export_dataset \
  --log-filter='resource.type="k8s_container"'

# Grant the sink's service account write access to the dataset (printed by the create command)
```

D. View and filter logs in Logs Explorer

```
gcloud logging read 'resource.type="gce_instance" AND severity>=ERROR' --limit=20
```

Console → Logging → Logs Explorer, query:

```
resource.type="k8s_container"
resource.labels.cluster_name="ace-demo-cluster"
severity>=WARNING
```

E. View log message details

Click any log entry in Logs Explorer to expand `jsonPayload`, `labels`, `resource`, `severity`, and `timestamp` fields.

F. Cloud Trace

Console → Trace → Trace List to view request latency waterfalls across services (auto-instrumented for App Engine/Cloud Run when the Trace API is enabled).

G. View GCP status

Check `https://status.cloud.google.com` for platform-wide incidents — useful to rule out a GCP-side outage before deep-diving your own app.

Section 18: Uptime Check & Alerting (End-to-End Lab)

Theory

An **uptime check** periodically probes an endpoint (HTTP/HTTPS/TCP) from multiple global locations and records success/failure. Paired with an **alerting policy**, this gives you an end-to-end "is my app up, and did anyone get notified" workflow — a foundational SRE practice and a favorite ACE exam scenario because it exercises Compute Engine, Monitoring, and IAM/notification channels together.

Hands-On Lab

Step 1 — Create a VM instance, allow HTTP

```
gcloud compute instances create uptime-demo-vm \
  --zone=asia-south1-a --machine-type=e2-small \
  --tags=http-server \
  --image-family=debian-12 --image-project=debian-cloud

gcloud compute firewall-rules create allow-http \
  --allow=tcp:80 --target-tags=http-server --source-ranges=0.0.0.0/0
```

Step 2 — Deploy a "Hello World" app

```
gcloud compute ssh uptime-demo-vm --zone=asia-south1-a --command='
  sudo apt-get update && sudo apt-get install -y nginx
  echo "Hello World from Uptime Demo" | sudo tee /var/www/html/index.html
'
```

Step 3 — Create an uptime check

```
gcloud monitoring uptime create hello-world-uptime-check \
  --resource-type=uptime-url \
  --resource-labels=host=$(gcloud compute instances describe uptime-demo-vm --zone=asia-south1-
  --path="/" --port=80 --check-interval=60s
```

Step 4 — Create an alert tied to the uptime check

```
cat > uptime-alert.json << 'EOF'
{
  "displayName": "Uptime Check Failure Alert",
  "conditions": [{
    "displayName": "Uptime check failed",
    "conditionThreshold": {
      "filter": "metric.type=\"monitoring.googleapis.com/uptime_check/check_passed\" AND resour
      "comparison": "COMPARISON_EQ",
      "thresholdValue": 0,
      "duration": "60s"
    }
  }],
  "notificationChannels": ["NOTIFICATION_CHANNEL_ID"]
```

```
}
EOF
gcloud alpha monitoring policies create --policy-from-file=uptime-alert.json
```

Create the email notification channel first if you haven't:

```
gcloud alpha monitoring channels create \
  --display-name="ACE Email Alerts" \
  --type=email --channel-labels=email_address=you@example.com
```

Step 5 — Stop the VM (or shut down the app) to trigger the alert

```
gcloud compute instances stop uptime-demo-vm --zone=asia-south1-a
```

Step 6 — Check email for the alert

Within a few minutes of the check interval, the configured email address receives an **incident notification**.

Step 7 — Observe the incident

Console → Monitoring → Alerting → Incidents — the incident shows as **Open**, with a timeline of the failing condition.

Step 8 — Start the app again

```
gcloud compute instances start uptime-demo-vm --zone=asia-south1-a
```

Step 9 — Close the incident & check the resolution email

Once the uptime check passes again, Cloud Monitoring auto-resolves the incident and sends a **"Incident resolved"** email. You can also manually acknowledge/close it from **Console → Monitoring → Alerting → Incidents → Close**.

Section 19: Lab — Cloud Logging (Hands-On)

Theory

Cloud Logging automatically ingests logs from most GCP services (GCE, GKE, Cloud Run, Cloud Functions, etc.) with **no agent required for platform-level logs**; the **Ops Agent** is needed on Compute Engine VMs to collect OS-level and application logs. Logs are queryable in real time via the Logs Explorer query language, and can be routed to sinks for long-term retention or downstream processing.

Hands-On Lab

Step 1 — Install the Ops Agent on a Compute Engine VM (for OS + app logs)

```
gcloud compute ssh uptime-demo-vm --zone=asia-south1-a --command='
  curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh
  sudo bash add-google-cloud-ops-agent-repo.sh --also-install
'
```

Step 2 — Generate some log entries

```
gcloud compute ssh uptime-demo-vm --zone=asia-south1-a --command='
  logger "Test log entry from ACE lab"
'
```

Step 3 — Query logs from the CLI

```
gcloud logging read 'resource.type="gce_instance" AND textPayload:"ACE lab"' --limit=10
```

Step 4 — Query logs in Console → Logs Explorer

```
resource.type="gce_instance"
resource.labels.instance_id="INSTANCE_ID"
```

Use the **histogram** view to spot spikes in log volume, and **severity** filters (`severity>=ERROR`) to isolate problems.

Step 5 — Create a saved query / log-based alert

Click **"Create alert"** directly from a Logs Explorer query to convert a filter into a real-time alerting policy without leaving the page.

Step 6 — Export logs to Cloud Storage for long-term retention

```
gcloud storage buckets create gs://YOUR_PROJECT_ID-log-archive --location=asia-south1

gcloud logging sinks create archive-sink \
  storage.googleapis.com/YOUR_PROJECT_ID-log-archive \
  --log-filter='resource.type="gce_instance"'
```

Section 20: Cloud Debugger

Theory

Cloud Debugger (historically part of the Stackdriver suite) lets you inspect the state of a **running application in production** — variables, call stack, at a specific line — **without stopping it or adding logging statements**. It's supported for **Java** (among a few other languages historically), which matters directly for the Java Spring Boot workloads used elsewhere in this guide. You set a "snapshot" at a line of code; the next time that line executes for any request, the debugger captures the full local variable state and call stack, then automatically removes itself.

> **Note**: Google has been deprecating/sunsetting the standalone Cloud Debugger product in recent years in favor of alternatives (e.g., Cloud Profiler for performance, or third-party APM). Verify current availability in the Console before relying on it for your specific exam version — this section documents the classic ACE-tested workflow.

Hands-On Lab

Step 1 — Enable the Debugger API

```
gcloud services enable clouddebugger.googleapis.com
```

Step 2 — Add the Debugger agent to your Java app

Add to `pom.xml`:

```
<dependency>
  <groupId>com.google.cloud</groupId>
  <artifactId>google-cloud-debugger-java-agent</artifactId>
  <version>LATEST</version>
</dependency>
```

Start the app with the agent attached:

```
java -agentpath:/path/to/cdbg_java_agent.so \
  -Dcom.google.cdbg.module=spring-pubsub-consumer \
  -Dcom.google.cdbg.version=v1 \
  -jar app.jar
```

Step 3 — Deploy the instrumented app (GKE, App Engine, or Compute Engine all work)

Use the same deploy flow as **Section 2/4/6** — the agent registers itself with the Debugger service on startup.

Step 4 — Set a Snapshot in Console

Console → Debug → Snapshots → Create Snapshot - Select the source file and line number to inspect (e.g., inside `PubSubHandler.accept()`). - Optionally add a **condition** (e.g., `event.getId() != null`) so it only triggers for matching requests.

Step 5 — Trigger the code path

```
gcloud pubsub topics publish java-fn-topic --message="trigger debugger snapshot"
```

Step 6 — Inspect the captured state

Once the line executes, the snapshot in Console populates with the full **local variable values** and **call stack** at that exact moment — all without pausing the live application or affecting other in-flight requests.

Step 7 — Set a Logpoint (alternative to a snapshot)

A **Logpoint** injects a log statement at a given line dynamically, without a redeploy — useful for adding temporary diagnostic logging to a production app.

Section 21: Managing Identity and Access Management (IAM)

Theory

GCP IAM follows the pattern: **who** (member — user, group, service account) gets **what role** (a bundle of permissions) on **which resource** (project, folder, org, or individual resource). Roles come in three flavors:

- **Basic roles** (Owner/Editor/Viewer): broad, legacy, generally avoided in production for least-privilege reasons.
- **Predefined roles**: curated by Google per service (e.g., `roles/storage.objectViewer`) — the recommended default.
- **Custom roles**: a hand-picked set of permissions when predefined roles are too broad or too narrow.

IAM policies are **additive only** (no explicit "deny" in the base policy, though **IAM Deny policies** now exist as a newer feature) and inherit down the resource hierarchy (Org → Folder → Project → Resource).

Hands-On Lab

A. View current IAM role assignments

```
gcloud projects get-iam-policy YOUR_PROJECT_ID --format=json
```

Or **Console → IAM & Admin → IAM** to see the member/role table with inheritance indicators.

B. Assign IAM roles to a user or Google Group

```
# To an individual user
gcloud projects add-iam-policy-binding YOUR_PROJECT_ID \
  --member="user:[email protected]" \
  --role="roles/compute.instanceAdmin.v1"

# To a Google Group (manages many users at once)
gcloud projects add-iam-policy-binding YOUR_PROJECT_ID \
  --member="group:devops-team@yourdomain.com" \
  --role="roles/container.developer"
```

C. Define a custom IAM role

```
cat > custom-role.yaml << 'EOF'
title: "GKE Pod Viewer Plus"
description: "View pods and their logs, nothing else"
stage: "GA"
includedPermissions:
  - container.pods.get
  - container.pods.list
  - logging.logEntries.list
EOF
```

```
gcloud iam roles create gkePodViewerPlus \
  --project=YOUR_PROJECT_ID --file=custom-role.yaml
```

D. Assign the custom role

```
gcloud projects add-iam-policy-binding YOUR_PROJECT_ID \
  --member="user:[email protected]" \
  --role="projects/YOUR_PROJECT_ID/roles/gkePodViewerPlus"
```

E. Remove a role binding

```
gcloud projects remove-iam-policy-binding YOUR_PROJECT_ID \
  --member="user:[email protected]" \
  --role="roles/container.developer"
```

Section 22: Managing Service Accounts

Theory

A **service account** is a special type of identity used by applications/VMs/functions (not humans) to call GCP APIs. Best practices tested on the ACE exam:

- **Limited privileges**: Grant a service account only the roles it needs (least privilege) — never `Owner`/`Editor` on a service account used by an app.
- **Attaching to VM instances**: A VM runs "as" its attached service account, and any code on it inherits that identity's permissions — so the SA attached at VM creation time effectively defines what the VM (and anything running on it) can do to other GCP resources.
- **Cross-project access**: A service account from Project A can be granted a role in Project B, allowing controlled cross-project resource access without duplicating credentials.

Hands-On Lab

A. Create a service account with limited privileges

```
gcloud iam service-accounts create ace-app-sa \
  --display-name="ACE App Service Account"

# Grant only what's needed — e.g., read objects from one bucket
gcloud storage buckets add-iam-policy-binding gs://YOUR_PROJECT_ID-images \
  --member="serviceAccount:[email protected]" \
  --role="roles/storage.objectViewer"
```

B. Assign a service account to a VM instance

```
gcloud compute instances create ace-app-vm \
  --zone=asia-south1-a \
  --service-account=[email protected] \
  --scopes=cloud-platform
```

`--scopes=cloud-platform` lets the underlying IAM role bindings (not OAuth scopes) fully govern access — the modern recommended pattern instead of narrowly-scoped legacy OAuth scopes.

C. Grant access to a service account in another project

```
# From Project2, grant Project1's service account access to a Project2 bucket
gcloud storage buckets add-iam-policy-binding gs://PROJECT2_BUCKET \
  --member="serviceAccount:[email protected]" \
  --role="roles/storage.objectViewer" \
  --project=PROJECT2_ID
```

D. Rotate / manage keys (if using external key-based auth instead of attached SAs)

```
gcloud iam service-accounts keys create key.json \
  --iam-account=[email protected]

gcloud iam service-accounts keys list \
```

```
  --iam-account=[email protected]

gcloud iam service-accounts keys delete KEY_ID \
  --iam-account=[email protected]
```

Prefer **Workload Identity** (GKE) or attached service accounts (Compute Engine) over downloadable JSON keys wherever possible — keys are a long-lived credential and a common source of leaks.

Section 23: Consolidated Service Account Lab — Limited Privileges, VM Assignment, Cross-Project Access

Theory

This section reinforces Section 22 with a single **end-to-end scenario** that exercises all three service-account concepts together — a realistic pattern for the ACE exam: an application VM in **Project1** needs to read files from a bucket owned by **Project2**, using a dedicated, least-privilege service account.

Hands-On Lab

Step 1 — Create a dedicated service account in Project1 (the "app" project)

```
gcloud iam service-accounts create cross-project-sa \
  --project=PROJECT1_ID \
  --display-name="Cross-Project Reader SA"
```

Step 2 (a) — Limit its privileges: grant only what's needed in Project1 itself

```
gcloud projects add-iam-policy-binding PROJECT1_ID \
  --member="serviceAccount:[email protected]" \
  --role="roles/logging.logWriter"
```

This lets the SA write its own application logs, and nothing else in Project1 — deliberately narrow.

Step 2 (b) — Assign the service account to a VM instance in Project1

```
gcloud compute instances create cross-project-vm \
  --project=PROJECT1_ID --zone=asia-south1-a \
  --service-account=[email protected] \
  --scopes=cloud-platform
```

Step 2 (c) — Grant the service account access to a resource in Project2

```
gcloud storage buckets add-iam-policy-binding gs://PROJECT2_SHARED_BUCKET \
  --member="serviceAccount:[email protected]" \
  --role="roles/storage.objectViewer" \
  --project=PROJECT2_ID
```

Step 3 — Verify from inside the VM

```
gcloud compute ssh cross-project-vm --project=PROJECT1_ID --zone=asia-south1-a --command='
  gcloud storage ls gs://PROJECT2_SHARED_BUCKET
'
```

This should succeed (list objects) even though the VM lives entirely in Project1 — proving the cross-project grant works — while any attempt to, say, delete a Compute Engine instance in Project2 would correctly fail (least privilege enforced).

Step 4 — Audit the final configuration

```
gcloud projects get-iam-policy PROJECT1_ID --flatten="bindings[].members" \
  --filter="bindings.members:cross-project-sa" --format="table(bindings.role)"

gcloud storage buckets get-iam-policy gs://PROJECT2_SHARED_BUCKET
```

Step 5 — Cleanup

```
gcloud compute instances delete cross-project-vm --project=PROJECT1_ID --zone=asia-south1-a --q
gcloud storage buckets remove-iam-policy-binding gs://PROJECT2_SHARED_BUCKET \
  --member="serviceAccount:[email protected]" \
  --role="roles/storage.objectViewer" --project=PROJECT2_ID
gcloud iam service-accounts delete [email protected] --project=PROJECT1_ID --quiet
```

End of GCP-ACE Advance — Part II Deep-Dive Lab Guide.